

A

EJERCICIO 1: CREACIÓN DE LA BASE DE DATOS (DDL)

ENUNCIADO:

Crea una base de datos llamada Ejercicio1_TuNombre y diseña las tablas necesarias para gestionar una pequeña red social con las siguientes especificaciones:

Usuarios: Se debe almacenar un identificador único, nombre (único), email (único) y fecha de registro.

Publicaciones: Los usuarios pueden crear publicaciones. De cada publicación se guarda el contenido, la fecha y quién la escribió. Si un usuario es eliminado, sus publicaciones deben desaparecer automáticamente.

Comentarios: Los usuarios pueden comentar las publicaciones. Se debe guardar el texto, la fecha, quién comenta y en qué publicación.

Likes: Los usuarios pueden valorar publicaciones. Se almacena la fecha del like y una valoración (nota del 1 al 5). Un usuario solo puede dar un like por publicación (clave primaria compuesta).

Seguimientos: Un usuario puede seguir a otros usuarios. Se debe guardar quién sigue a quién y la fecha. **Restricción:** Un usuario no puede seguirse a sí mismo.

Requisitos: Definir PKs, FKs y tipos de datos adecuados.

-- 1. PREPARACIÓN DE LA BASE DE DATOS

```
DROP DATABASE IF EXISTS Ejercicio1_Solucion;
CREATE DATABASE Ejercicio1_Solucion CHARACTER SET utf8mb4 COLLATE
utf8mb4_spanish_ci;
USE Ejercicio1_Solucion;
```

-- 2. TABLA USUARIOS (La tabla "Padre", no depende de nadie)

```
CREATE TABLE usuarios (
    id_usuario INT NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(50) NOT NULL UNIQUE,
    fecha_registro DATE NOT NULL,
    CONSTRAINT pk_usuarios PRIMARY KEY (id_usuario)
);
```

-- 3. TABLA PUBLICACIONES (Hija de Usuarios)

```
CREATE TABLE publicaciones (
    id_publicacion INT NOT NULL AUTO_INCREMENT,
    contenido VARCHAR(100) NOT NULL,
    fecha_publicacion DATE NOT NULL,
    -- Importante: Columna para saber QUIÉN publicó esto
    id_usuario_autor INT NOT NULL,

    CONSTRAINT pk_publicaciones PRIMARY KEY (id_publicacion),

    -- Relación: Si borras al usuario, se borran sus publicaciones (Cascade)
    CONSTRAINT fk_publicaciones_usuarios FOREIGN KEY (id_usuario_autor)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE
);
```

-- 4. TABLA COMENTARIOS (Hija de Usuarios y Publicaciones)

```
CREATE TABLE comentarios (
    id_comentario INT NOT NULL AUTO_INCREMENT,
    texto_comentario VARCHAR(100) NOT NULL,
    fecha_comentario DATE NOT NULL,

    -- Necesitamos saber QUIÉN comenta y DÓNDE comenta
    id_usuario_autor INT NOT NULL,
    id_publicacion_destino INT NOT NULL,
```

```

CONSTRAINT pk_comentarios PRIMARY KEY (id_comentario),

-- FK 1: El autor del comentario
CONSTRAINT fk_comentarios_usuarios FOREIGN KEY (id_usuario_autor)
    REFERENCES usuarios (id_usuario) ON DELETE CASCADE,

-- FK 2: La publicación comentada
CONSTRAINT fk_comentarios_publi FOREIGN KEY (id_publicacion_destino)
    REFERENCES publicaciones (id_publicacion) ON DELETE CASCADE
);

-- 5. TABLA LIKES (Relación N:M entre Usuarios y Publicaciones)
CREATE TABLE likes (
    -- Aquí no suele hacer falta un ID propio, la clave es la pareja
    usuario+publi
    id_usuario INT NOT NULL,
    id_publicacion INT NOT NULL,
    fecha_like DATE NOT NULL,
    nota_valoracion CHAR(1) NOT NULL, -- Ojo, CHAR(1) guarda '1', '5', etc.

    -- PK Compuesta: Un usuario solo puede dar 1 like por publicación
    CONSTRAINT pk_likes PRIMARY KEY (id_usuario, id_publicacion),

    -- Validación: Que la nota sea un número del 1 al 5 (opcional pero
    recomendable)
    CONSTRAINT chk_valoracion CHECK (nota_valoracion BETWEEN '1' AND '5'),

    -- FKs
    CONSTRAINT fk_likes_usuarios FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_likes_publi FOREIGN KEY (id_publicacion)
        REFERENCES publicaciones (id_publicacion) ON DELETE CASCADE
);

-- 6. TABLA SEGUIMIENTOS (Relación Reflexiva N:M)
CREATE TABLE seguimientos (
    id_seguimiento INT NOT NULL AUTO_INCREMENT, -- Opcional, podrías usar PK
    compuesta
    id_follower INT NOT NULL, -- El que sigue
    id_followed INT NOT NULL, -- El seguido
    fecha_seguimiento DATE NOT NULL,

    CONSTRAINT pk_seguimientos PRIMARY KEY (id_seguimiento),

    -- Ambas FK apuntan a la tabla USUARIOS
    CONSTRAINT fk_seg_follower FOREIGN KEY (id_follower)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_seg_followed FOREIGN KEY (id_followed)
        REFERENCES usuarios (id_usuario) ON DELETE CASCADE,

    -- Validación: No te puedes seguir a ti mismo
    CONSTRAINT chk_no_autoseguimiento CHECK (id_follower <> id_followed),

    -- Validación extra: No puedes seguir dos veces a la misma persona
    CONSTRAINT uq_seguimiento_unico UNIQUE (id_follower, id_followed)
);

```

Ejercicio 2

ENUNCIADO:

Dada la sentencia SQL inicial que crea la tabla coches (matricula, marca_modelo), realiza las siguientes modificaciones usando ALTER TABLE:

Renombra la tabla coches a vehiculos.

Renombra la columna marca_modelo a marca.

Añade una nueva columna llamada modelo (VARCHAR 50, no nula).

-- 1. Preparación limpia

```
DROP DATABASE IF EXISTS Ejercicio2_Solucion;
```

```
CREATE DATABASE Ejercicio2_Solucion CHARACTER SET utf8mb4;
```

```
USE Ejercicio2_Solucion;
```

-- 2. Crear la tabla original (Solo crear, nada más)

```
CREATE TABLE coches (
```

```
    matricula CHAR(7) NOT NULL,
```

```
    marca_modelo VARCHAR(50) NOT NULL,
```

```
    CONSTRAINT pk_coches PRIMARY KEY (matricula)
```

```
);
```

-- 3. Modificaciones (UNA POR UNA)

-- A) Renombrar la tabla de 'coches' a 'vehiculos'

```
ALTER TABLE coches RENAME TO vehiculos;
```

-- B) Renombrar la columna 'marca_modelo' a 'marca'

```
ALTER TABLE vehiculos RENAME COLUMN marca_modelo TO marca;
```

-- C) Añadir la nueva columna 'modelo'

-- (Usamos VARCHAR porque un modelo suele ser texto, ej: "Fiesta", "Ibiza")

```
ALTER TABLE vehiculos ADD COLUMN modelo VARCHAR(50) NOT NULL;
```

-- 4. Comprobación final

```
DESCRIBE vehiculos;
```

Ejercicio 3

En una BD con una tabla usuarios, realiza lo siguiente:

Crea un usuario 'usuarios_add' con contraseña 'usuarios' que pueda conectarse desde CUALQUIER HOST.

Crea un Rol llamado 'escritura'.

Asigna al rol 'escritura' los permisos para INSERTAR, ACTUALIZAR y BORRAR datos sobre la tabla usuarios.

Asigna el rol 'escritura' al usuario 'usuarios_add'.

-- 1. Preparación limpia

```
DROP DATABASE IF EXISTS Ejercicio3_Solucion;
```

```
CREATE DATABASE Ejercicio3_Solucion CHARACTER SET utf8mb4;
```

```
USE Ejercicio3_Solucion;
```

-- 2. Crear la tabla usuarios

```
CREATE TABLE usuarios (
```

```
    id_usuario VARCHAR(50) NOT NULL,
```

```
    password VARCHAR(50) NOT NULL,
```

```
    CONSTRAINT pk_usuarios PRIMARY KEY (id_usuario)
```

```
);
```

-- 3. Crear el usuario del sistema

-- Usamos '%' para que pueda conectarse desde cualquier host

```
CREATE USER 'usuarios_add'@'%' IDENTIFIED BY 'usuarios';
```

-- 4. Asignar permisos

-- Le damos permiso de INSERTAR datos en la tabla 'usuarios'

```
GRANT INSERT ON Ejercicio3_Solucion.usuarios TO 'usuarios_add'@'%';
```

-- 5. Comprobación (Opcional)

```
SHOW GRANTS FOR 'usuarios_add'@'%';
```